Informe Laboratorio 3

Sección x

Alumno x
e-mail: oseas.poveda@mail.udp.cl

Octubre de 2026

Índice

1. Descripción de actividades **2**

2. Desarrollo de actividades según criterio de rúbrica **2**
2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio 2
2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión 3
2.3. Genera el hash de la contraseña desde la consola del navegador 3
2.4. Intercepta el tráfico login con BurpSuite 4
2.5. Realiza el intento de login 5
2.6. Identifica las políticas de privacidad o seguridad 7
2.7. Demuestra 4 conclusiones sobre la seguridad 8

1. Descripción de actividades

Su objetivo será auditar la implementación de algoritmos hash aplicados a contraseñas en páginas web desde el lado del cliente, así como evaluar la efectividad de estas medidas contra ataques de tipo Pass the Hash (PtH). Para llevar a cabo esta auditoría, deberá registrarse en un sitio web y crear una cuenta, ingresando una contraseña específica para realizar las pruebas.

Al concluir la tarea, es importante que modifique su contraseña por una diferente para garantizar su seguridad.

Dado que la cantidad de sitios chilenos que utilizan hash es limitada, se permite realizar esta tarea en cualquier sitio web a nivel mundial. En este sentido, realice las siguientes actividades:

- Identificación del algoritmo de hash utilizado para las contraseñas al momento del registro en el sitio.
- Identificación del algoritmo de hash utilizado para las contraseñas al momento de iniciar sesión.
- Generación del hash de la contraseña desde la consola del navegador, partiendo de la contraseña en texto plano.
- Interceptación del tráfico de login utilizando BurpSuite desde su equipo.
- Realización de un intento de login, modificando una contraseña incorrecta por el hash obtenido en el punto anterior.
- Descripción de las políticas de privacidad o seguridad relacionadas con las contraseñas, incluyendo un enlace a las mismas.
- Cuatro conclusiones sobre la seguridad o vulnerabilidad de la implementación observada.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio

Para esta actividad se seleccionó el sitio `LinuxQuestions.org`, un foro en el cual se hace la implemnetacion de hash a las contraseñas por medio del cliente. Al crear una cuenta nueva, se inspeccionó el código fuente del formulario de registro mediante las herramientas de desarrollador del navegador (pestaña Elements). Dentro del atributo onsubmit del formulario se encontró una llamada a la función md5hash(), cargada desde el script vbulletin_md5.js. Esto indica que la contraseña ingresada por el usuario es transformada a un hash MD5 directamente en el navegador, antes de ser enviada al servidor.

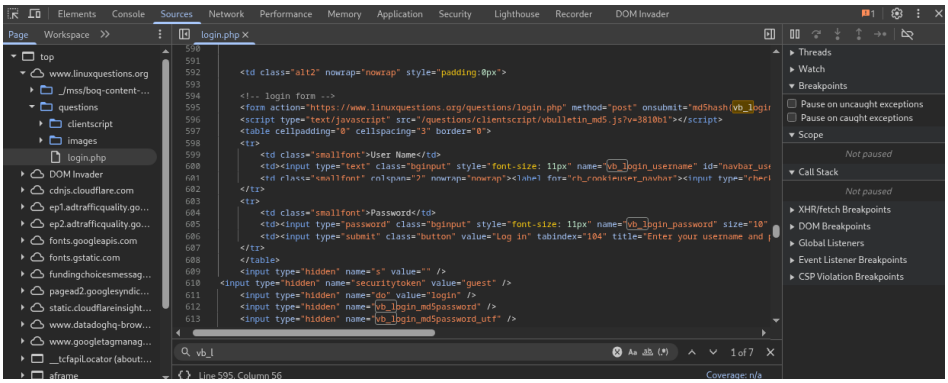


Figura 1: Formulario de registro del sitio web

2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión

Se repitió el mismo procedimiento en el formulario de inicio de sesión. Nuevamente se identificó el uso del algoritmo MD5, ocupando la misma función JavaScript del lado del cliente. La función toma el valor en texto plano del campo vb_login_password y genera dos campos ocultos adicionales (vb_login_md5password y vb_login_md5password_utf) que contienen el hash resultante, siendo estos los que efectivamente se transmiten al servidor en la solicitud HTTP.

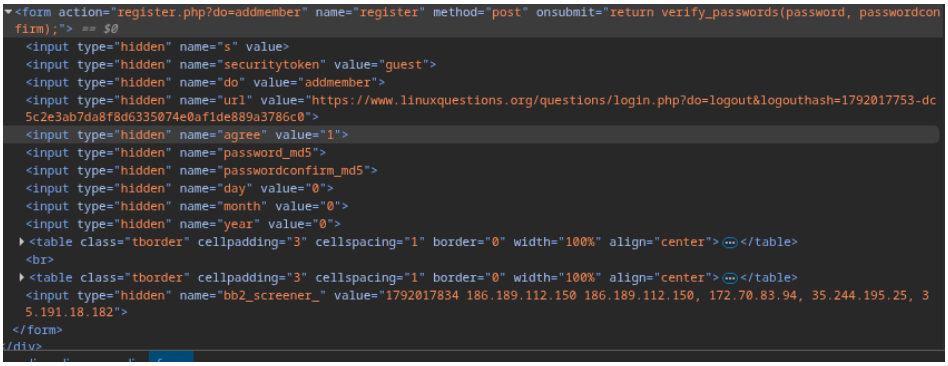


Figura 2: Evidencia del uso del algoritmo MD5 en el formulario de inicio de sesión

2.3. Genera el hash de la contraseña desde la consola del navegador

Para comprobar de forma independiente el comportamiento observado, se utilizó la consola de desarrollador del navegador en la propia página de login, donde ya se encuentra cargada la función de hashing del sitio. Se invocó manualmente dicha función utilizando como entrada la contraseña de prueba en texto plano, obteniéndose como salida una cadena hexadecimal de 32 caracteres. Al comparar este resultado con el valor transmitido en la solicitud de red,

ambos coincidieron exactamente, confirmando que el hash se calcula enteramente en el cliente y que el servidor recibe únicamente dicho valor.

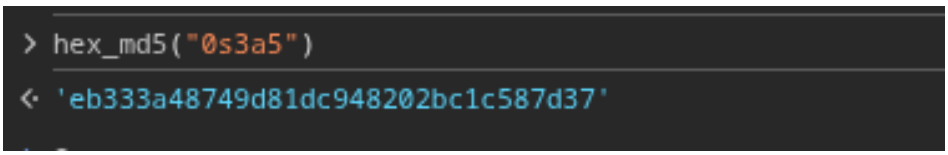


Figura 3: Generación manual del hash MD5 desde la consola del navegador

2.4. Intercepta el tráfico login con BurpSuite

Se configuró Burp Suite como proxy del navegador y se instaló su certificado para interceptar tráfico HTTPS. Con la función Intercept activada, se inició sesión en el sitio y se capturó la solicitud POST dirigida a login.php. Dentro del cuerpo de la solicitud se verificó que la contraseña no viaja en texto plano, sino codificada dentro del parámetro vb_login_md5password, cuyo valor corresponde exactamente al hash generado en el paso anterior.



Figura 4: Tráfico de login interceptado mostrando el parámetro con el hash MD5

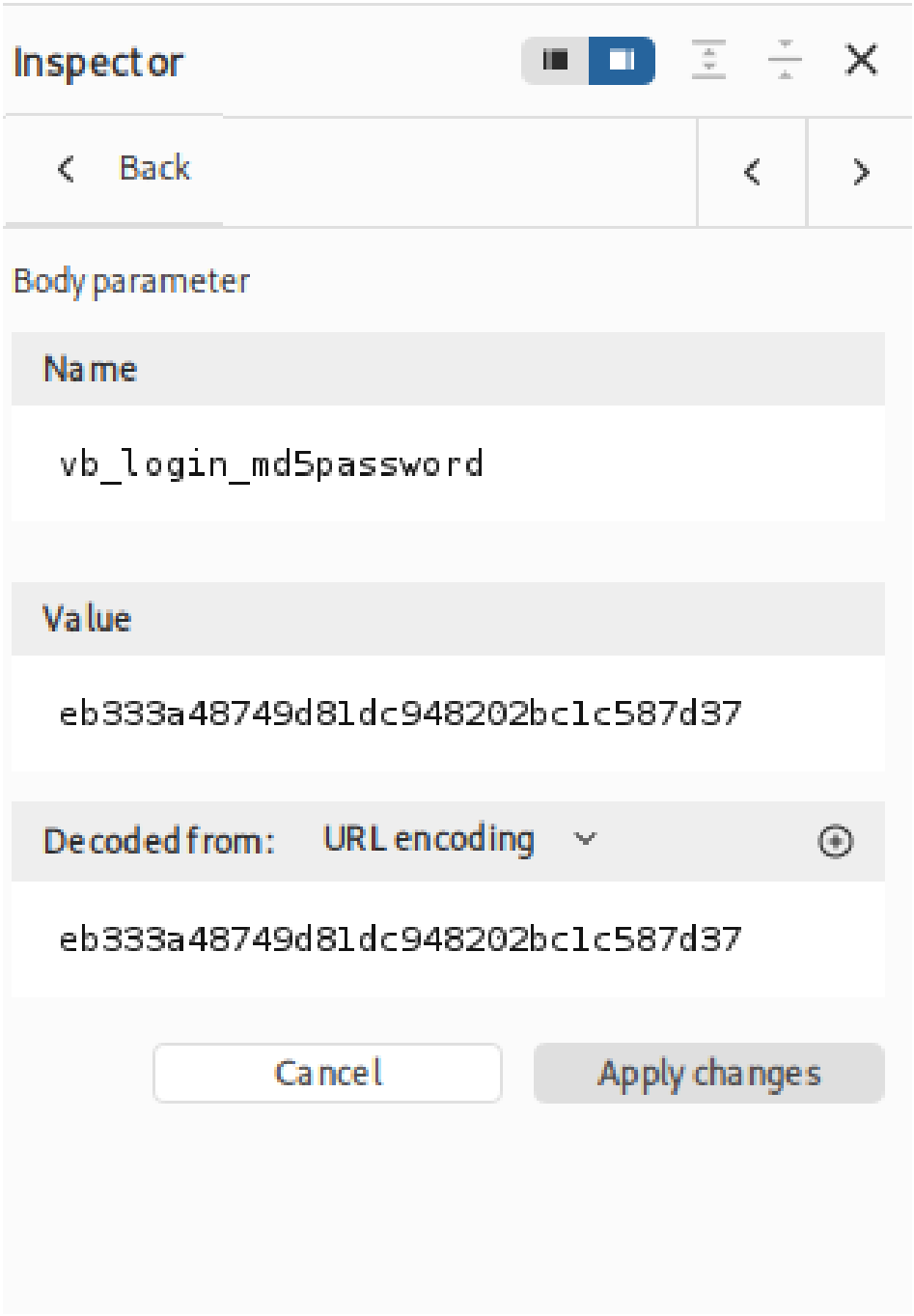


Figura 5: Valor original del parámetro antes de ser modificado

2.5. Realiza el intento de login

Con la solicitud capturada, se envió al módulo Repeater de Burp Suite para evaluar si el servidor es vulnerable a un ataque de tipo Pass the Hash, es decir, si acepta el hash como credencial válida sin requerir en ningún momento la contraseña en texto plano.

En primera instancia se alteró deliberadamente el valor del parámetro vb_login_md5password, simulando el envío de una credencial incorrecta, (se cambio para que al final del has diga op67). El servidor respondió denegando la autenticación, sin entregar cookies de sesión.

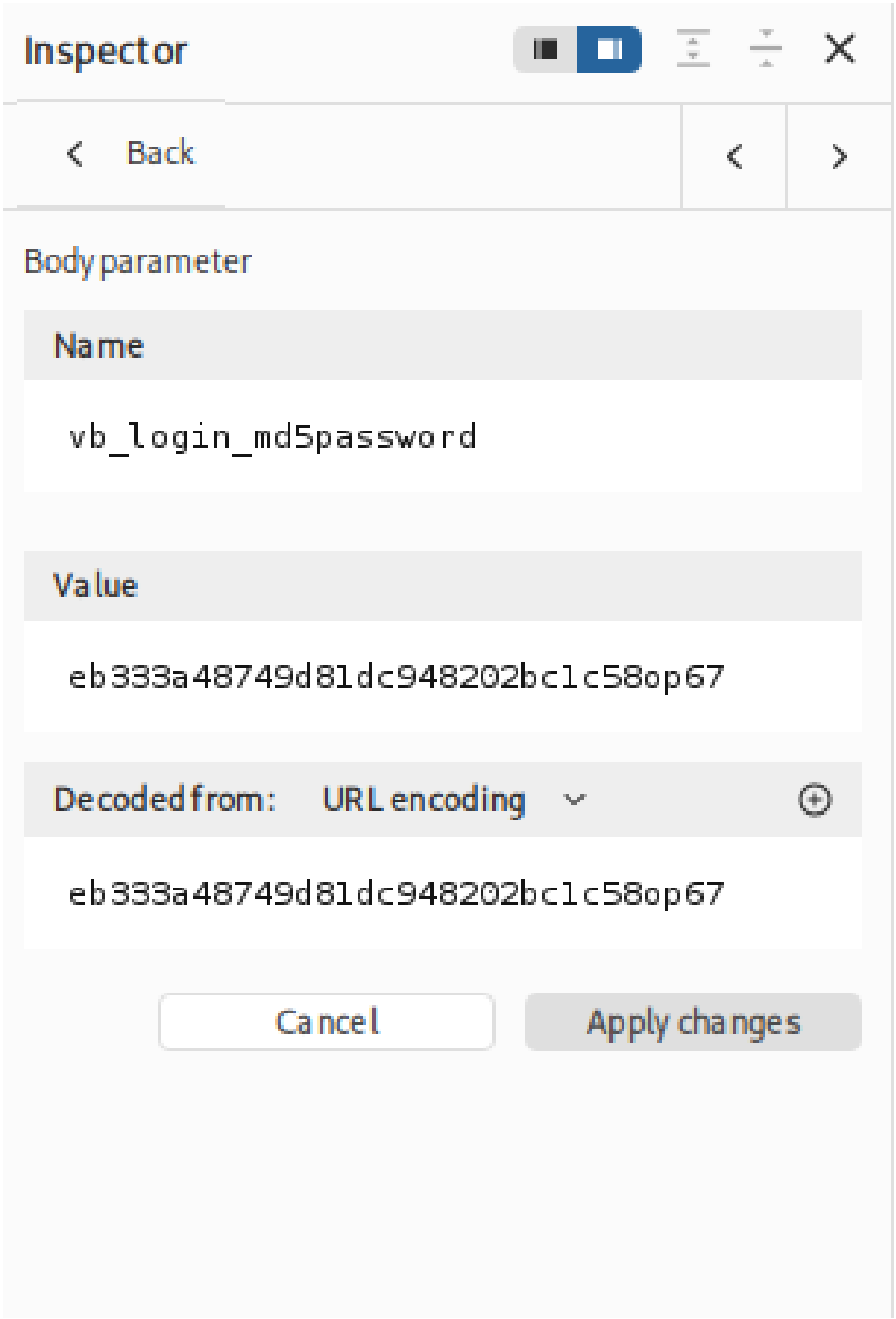


Figura 6: Modificación del parámetro de hash con un valor incorrecto

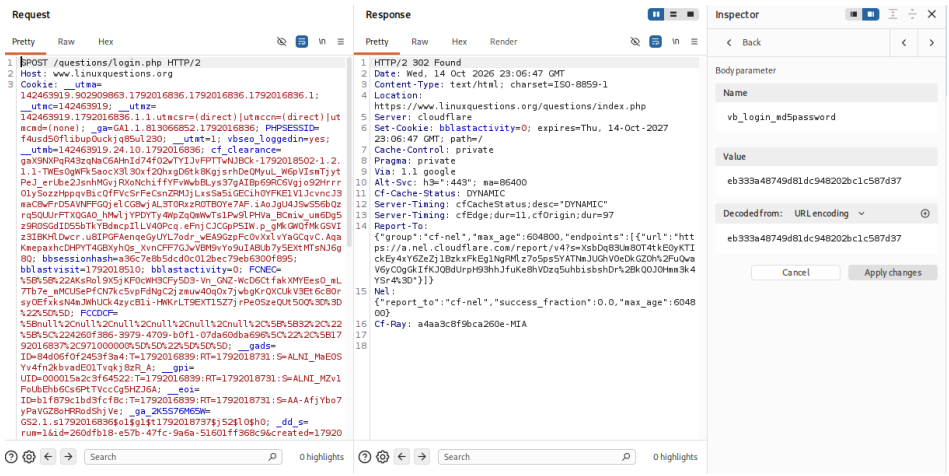


Figura 7: Respuesta del servidor denegando el acceso

Posteriormente, se restauró el valor original y correcto del hash, enviando la solicitud nuevamente sin incluir en ningún momento la contraseña en texto plano. En esta ocasión, el servidor respondió con código de estado 200 OK, entregando las cookies de sesión correspondientes a un usuario autenticado. Esto confirma que el backend valida directamente el hash recibido, lo que constituye una vulnerabilidad de tipo Pass the Hash: basta con interceptar o filtrar el hash, sin conocer la contraseña original, para suplantar la identidad del usuario.

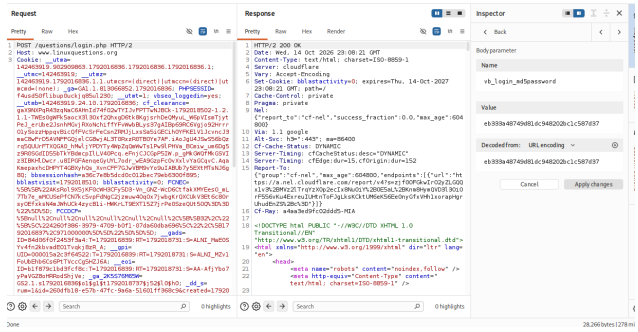


Figura 8: Autenticación exitosa restaurando el hash correcto

2.6. Identifica las políticas de privacidad o seguridad

LinuxQuestions.org documenta sus políticas de gestión y tratamiento de credenciales en sus secciones públicas. Las normativas pueden consultarse en los siguientes enlaces:

- **Política de Privacidad:** `https://www.linuxquestions.org/privacy.php`
- **Reglas de Registro:** `https://www.linuxquestions.org/questions/register.php`

En términos generales, el sitio no detalla públicamente el mecanismo criptográfico exacto con el que almacena las contraseñas en su base de datos, limitándose a declaraciones genéricas

sobre el resguardo de la información de los usuarios. El formulario de registro exige una longitud mínima de caracteres, pero no impone explícitamente el uso de mayúsculas, símbolos o números, por lo que la robustez de la contraseña depende principalmente de la elección del propio usuario. Tampoco se ofrece un mecanismo de autenticación de múltiples factores (2FA/MFA) para cuentas estándar. El único control de seguridad consistentemente aplicado es el cifrado del canal de transporte mediante HTTPS/TLS, el cual protege el tráfico frente a observadores externos en la red, aunque no corrige las debilidades del hashing realizado en el cliente.(no debi dejar mi correo real en la pagina)

2.7. Demuestra 4 conclusiones sobre la seguridad

1. **El hashing en el cliente no reemplaza la confidencialidad de la contraseña:** dado que el servidor valida directamente el hash MD5 recibido, dicho valor cumple lo mismo que deberia hacer la contraseña en texto plano. Cualquier persona que logre interceptar este parámetro puede reutilizarlo para autenticarse sin necesidad de conocer la contraseña original, tal lo hice ne el informe mas arriba.

2. **MD5 es un algoritmo criptográficamente débil para este propósito:** al ser una función de hash rápida y sin mecanismo de fortalecimiento, MD5 es altamente susceptible a ataques de fuerza bruta acelerados por hardware gráfico, lo que facilita recuperar la contraseña original si el hash llega a filtrarse.

3. **No existe un mecanismo de reto dinámico:** el sistema no genera un valor aleatorio distinto (salt o peper por sesion) que se combine con la contraseña en cada intento de inicio de sesión. Como el hash enviado es siempre idéntico para una misma contraseña, el esquema queda expuesto a ataques de repetición (replay) a nivel de aplicación.

4. **El diseño contradice las recomendaciones actuales de la industria:** organismos como OWASP y NIST desaconsejan delegar operaciones criptográficas sensibles en el cliente, ya que este entorno no es confiable y puede ser manipulado por el propio usuario o un atacante. La práctica recomendada es transmitir la contraseña en texto plano exclusivamente a través de un canal cifrado y realizar el hashing en el servidor utilizando algoritmos lentos y con salt por usuario, como bcrypt, Argon2id o PBKDF2, diseñados específicamente para dificultar ataques de fuerza bruta.